GEWN — Feature Specification

Project GEWN | AI Desktop Copilot System
Document Type: Product Feature Specification
Audience: Engineers, Product Collaborators, Startup Team
Status: Living Document — v1.1

Overview

GEWN is an AI-native desktop copilot that combines conversational intelligence, autonomous task execution, screen-aware context, voice interaction, persistent memory, and workflow automation into a single unified system. It is designed to operate as an ambient intelligence layer on top of the user's existing desktop environment — not as a replacement for tools, but as the intelligent layer that connects and operates them.

This document defines all planned features, organized by capability domain. Each feature includes its purpose, user impact, and technical implementation direction. Features are classified as **Core** (planned for initial release) or **Future** (post-launch or experimental).

Feature Status Legend

Tag	Meaning
`[CORE]`	Required for v1 launch
`[FUTURE]`	Post-launch / experimental

---

1. Conversational AI

1.1 Natural Language Chat Interface `[CORE]`

What it does: Provides a persistent chat interface for issuing commands, asking questions, and receiving AI-generated responses in natural language.

Why it matters: Replaces rigid command-line and menu-driven UX with an intuitive interaction model accessible to all users.

Implementation:

- LLM backends: OpenAI GPT-4o, Gemini 1.5 Pro, or local models via Ollama
- Streaming response rendering with token-by-token output
- Context injection from active sessions, memory, and screen state
- LangChain for chain-of-thought orchestration and tool routing

---

1.2 Context-Aware Responses `[CORE]`

What it does: GEWN understands what the user is currently doing — active app, open files, screen content, cursor position, recent activity — and factors this into every response.

Why it matters: Transforms GEWN from a generic chatbot into a proactive assistant that gives relevant help without the user having to explain their situation.

Implementation:

- Active window detection via OS accessibility APIs
- Screenshot pipeline with multimodal LLM analysis
- Context enrichment layer injected into each prompt

1.3 Autonomous AI Agents `[CORE]`

What it does: Executes multi-step tasks automatically when given a high-level goal.

> *"Organize my AI project folder and summarize all PDFs inside it."*

Why it matters: Moves GEWN beyond reactive Q&A into proactive execution — a true AI operator.

Implementation:

- LangGraph for stateful multi-step agent workflows
- Tool-calling agents with retry, error recovery, and checkpointing

- Task planner with subtask decomposition
- Human-in-the-loop approval gates for risky operations

1.4 Multi-Agent System `[CORE]`

What it does: Routes complex tasks to specialized sub-agents, each optimized for a specific domain.

Agent	Responsibility
Coding Agent	Code generation, debugging, explanation
Research Agent	Web search, PDF summarization, synthesis
Automation Agent	File ops, app control, browser tasks
Memory Agent	Stores and retrieves user context
Scheduling Agent	Calendar management, reminders, task queues

Why it matters: Enables modular scalability — new agents can be added without modifying the core orchestration layer.

Implementation:

- Agent router with intent classification
- Modular prompt templates per agent domain
- Shared memory bus for inter-agent context passing

---

2. Voice Interface

2.1 Wake Word Activation `[CORE]`

What it does: Allows users to invoke GEWN hands-free using a configurable wake word (default: *"Hey GEWN"*).

Why it matters: Enables ambient, zero-friction interaction without switching focus from the active task.

Implementation:

- Porcupine wake-word engine (on-device, low-latency)
- Custom wake-word training support
- Configurable sensitivity and activation modes

2.2 Speech-to-Text [CORE]

What it does: Converts spoken language into structured commands and queries.

Why it matters: Reduces friction for voice-first workflows and accessibility use cases.

Implementation:

- OpenAI Whisper API for high-accuracy cloud transcription
- Local STT fallback (Whisper.cpp) for privacy mode
- Audio streaming with interim transcript display

2.3 Text-to-Speech [CORE]

What it does: GEWN responds in natural-sounding voice.

Why it matters: Creates a conversational, ambient experience instead of a screen-only assistant.

Implementation:

- ElevenLabs for high-quality, low-latency synthesis
- Coqui TTS as open-source local fallback
- Emotion-aware prosody (pacing, tone variation) based on response type

---

2.4 Real-Time Voice Mode [FUTURE]

What it does: Enables continuous, real-time voice conversations with GEWN — including interruption and barge-in support.

Why it matters: Makes GEWN feel like a genuine real-time AI companion, not a command-response bot.

Implementation:

- WebSocket-based bidirectional audio streaming
- VAD (Voice Activity Detection) for turn detection
- Interruption handling with graceful response cancellation

---

3. Desktop Automation

3.1 File Management `[CORE]`

What it does: Creates, renames, moves, organizes, and deletes files and folders via natural language commands.

Why it matters: Automates repetitive filesystem tasks that consume disproportionate cognitive load.

Implementation:

- Python `pathlib` and `shutil` for cross-platform operations
- Approval confirmation for destructive actions (delete, overwrite)
- Dry-run mode for previewing changes before execution

3.2 Application Control `[CORE]`

What it does: Opens, closes, switches, and orchestrates applications on the user's desktop.

Why it matters: Enables workflow automation across the user's tool ecosystem without manual app switching.

Implementation:

- Electron shell APIs and PyAutoGUI for app launch/focus
- Native OS integrations (Windows COM, macOS AppleScript/JXA)
- Window state detection (foreground, minimized, active)

3.3 Browser Automation `[CORE]`

What it does: Navigates websites, extracts information, fills forms, and automates browser-based research.

Why it matters: The majority of productivity workflows happen in browsers. Browser automation unlocks the web as an execution surface.

Implementation:

- Playwright for reliable cross-browser automation
- Browser agents with retry/fallback logic
- Anti-bot detection awareness (rate limiting, human-like delays)

3.4 Workflow Automation `[CORE]`

What it does: Executes saved, reusable workflow sequences triggered by command or schedule.

> *"Prepare my coding workspace"* → Opens VS Code, loads last project, opens relevant docs, starts local server.

Why it matters: Eliminates setup friction and enables repeatable daily routines.

Implementation:

- Event-driven workflow engine (trigger → action → condition)
- YAML-based workflow definition format
- Workflow recorder for capturing user action sequences

---

4. Vision & Screen Awareness

4.1 Screen Understanding `[CORE]`

What it does: Analyzes the user's screen in real time to provide contextually relevant assistance.

Why it matters: Allows GEWN to see what the user sees and respond with grounded, accurate help — no copy-pasting required.

Implementation:

- Periodic screenshot capture (configurable interval or on-demand)
- Multimodal LLM analysis (GPT-4 Vision, Gemini Vision)
- Differential screen analysis to detect meaningful changes

4.2 OCR Text Extraction `[CORE]`

What it does: Extracts text from any visible UI element, image, or screenshot.

Why it matters: Makes visual information machine-readable and actionable by the AI layer.

Implementation:

- PaddleOCR / Tesseract OCR as primary engines
- Region-of-interest cropping for targeted extraction
- Structured output parsing for tables and code blocks

4.3 Coding Error Detection `[CORE]`

What it does: Detects programming errors, warnings, and stack traces visible on screen and proposes fixes.

Why it matters: Closes the loop between seeing an error and understanding how to fix it — without switching context to a chat window.

Implementation:

- OCR extraction + semantic error analysis
- Language-aware fix suggestions (Python, JS, Rust, etc.)
- Direct IDE integration via Language Server Protocol (LSP)

4.4 Visual Overlay Guidance `[FUTURE]`

What it does: Renders transparent overlays on the user's screen to highlight errors, explain UI elements, and guide multi-step workflows.

Why it matters: Creates an interactive assistance layer on top of any application without modifying it.

Implementation:

- Electron transparent overlay window (always-on-top, click-through)
- SVG annotations rendered over targeted screen regions
- Step-by-step guidance choreography engine

---

5. Memory System

5.1 Short-Term Session Memory `[CORE]`

What it does: Maintains full context across a single working session — conversation history, active tasks, intermediate results.

Why it matters: Enables coherent multi-turn interactions without the user having to repeat themselves.

Implementation:

- Redis-backed session store with TTL expiry
- Rolling context window management (summarization when near token limits)
- Conversation thread isolation per task

5.2 Long-Term Semantic Memory `[CORE]`

What it does: Persists user preferences, past workflows, project context, and interaction patterns across sessions.

Why it matters: Makes GEWN progressively more personalized and useful the longer it is used.

Implementation:

- Embedding-based semantic storage (OpenAI `text-embedding-3-small`)
- Vector database: Qdrant or Chroma for fast similarity search

- Automatic memory consolidation and deduplication

5.3 Workflow Memory & Habit Learning `[CORE]`

What it does: Observes and learns recurring user workflows to enable proactive suggestions and automation.

Why it matters: Moves GEWN from reactive execution to anticipatory intelligence.

Implementation:

- Behavioral pattern detection from activity logs
- Workflow graph construction with frequency weighting
- Proactive suggestion engine with confidence thresholds

5.4 AI Second Brain `[FUTURE]`

What it does: A searchable, AI-indexed knowledge store for notes, screenshots, bookmarks, ideas, and research.

Why it matters: Turns GEWN into a persistent external memory system — a true digital second brain.

Implementation:

- RAG (Retrieval-Augmented Generation) pipeline
- Automatic ingestion from clipboard, screenshots, and browser
- Semantic search with natural language queries

---

6. Productivity & Workflow

6.1 Smart Workspace Setup `[CORE]`

What it does: Automatically configures the user's working environment — opens IDEs, browser tabs, documentation, terminals — based on the active project or time of day.

Why it matters: Eliminates the 5–10 minutes of daily setup overhead and context-switching friction.

Implementation:

- Workspace profile definitions (project-based, time-based)
- Automation script executor with dependency awareness

6.2 Task & Schedule Management `[CORE]`

What it does: Manages reminders, recurring tasks, deadlines, and schedules via natural language.

Why it matters: Integrates task management into the same interface as all other GEWN interactions.

Implementation:

- Calendar API integrations (Google Calendar, Outlook)
- Natural language date/time parsing
- Background scheduling engine with notification delivery

6.3 Smart File Organization `[CORE]`

What it does: Organizes files semantically — auto-tags, categorizes, and suggests folder structures based on content.

Why it matters: Reduces cognitive overhead of file management and improves retrieval.

Implementation:

- LLM-based file content analysis and tagging
- Semantic categorization rules (configurable by user)
- Non-destructive organization with preview before execution

6.4 Research Assistant `[CORE]`

What it does: Summarizes PDFs, extracts key insights, cross-references sources, and generates structured research notes.

Why it matters: Accelerates literature review, competitive research, and information synthesis.

Implementation:

- RAG architecture with document chunking and embedding
- Web extraction pipeline (Playwright + BeautifulSoup)
- Structured output: summaries, bullet points, citations

6.5 Productivity Analytics [FUTURE]

What it does: Tracks focus time, app usage, coding sessions, and workflow patterns. Surfaces optimization insights.

Why it matters: Gives users data-driven visibility into their own productivity.

Implementation:

- Background activity monitor (OS-level hooks)
- Timeline visualization with session tagging
- Weekly digest reports with AI-generated recommendations

6.6 Smart Notification Filtering [FUTURE]

What it does: Classifies incoming notifications by urgency and relevance, suppressing low-priority interruptions during focus sessions.

Why it matters: Reduces context-switching and protects deep work time.

Implementation:

- Notification classification model (urgency, sender, content)
- Focus mode integration with Do Not Disturb escalation rules

---

6.7 Meeting Assistant [FUTURE]

What it does: Transcribes meetings in real time, generates structured summaries, and extracts action items.

Why it matters: Eliminates manual note-taking and ensures nothing important is missed.

Implementation:

- Real-time audio transcription pipeline (Whisper)
- Summarization and action-item extraction via LLM
- Integration with calendar and task management systems

---

7. Plugin & Integration System

7.1 Third-Party Plugin Support `[FUTURE]`

What it does: Exposes a developer SDK for extending GEWN with custom agents, commands, and UI panels.

Why it matters: Creates an ecosystem where the community can extend GEWN for specialized use cases.

Implementation:

- Plugin manifest format (JSON/YAML)
- Event hook system (on-message, on-screen-change, on-task-complete)
- Sandboxed plugin execution environment

7.2 Service Integrations `[CORE]`

What it does: Connects GEWN to popular productivity and developer tools.

Supported integrations: GitHub, Notion, Gmail, Slack, Discord, Google Calendar, Jira, Linear

Why it matters: GEWN becomes the single interface for orchestrating the user's entire tool stack.

Implementation:

- OAuth 2.0 for secure authorization
- REST and Webhook-based event subscriptions
- Unified integration abstraction layer

7.3 Plugin Marketplace `[FUTURE]`

What it does: A curated marketplace for community-built GEWN plugins and integrations.

Why it matters: Enables community-driven growth and long-tail use case coverage.

Implementation:

- Plugin registry backend with versioning and signing
- In-app marketplace UI with install/update/remove lifecycle

---

8. Security & Privacy

8.1 Permission-Based Action Confirmation `[CORE]`

What it does: Presents explicit confirmation dialogs before executing irreversible or high-risk operations (deletes, system changes, external API calls).

Why it matters: Prevents accidental or unintended automation damage.

Implementation:

- Risk classifier (low / medium / high) per action type
- Configurable auto-approve threshold for trusted operation classes
- Audit log of all confirmed and denied actions

8.2 Encrypted Local Storage `[CORE]`

What it does: All memory, preferences, and user data are encrypted at rest.

Why it matters: Protects sensitive personal and professional data from unauthorized access.

Implementation:

- AES-256 encryption for vector stores and session data
- Secure vault for credentials and API keys (OS keychain integration)

8.3 Privacy Mode (Local-Only) [CORE]

What it does: Enables fully offline AI execution using local language models — no data leaves the device.

Why it matters: Supports privacy-sensitive workflows and air-gapped environments.

Implementation:

- Ollama runtime for local LLM inference (Llama 3, Mistral, Phi-3)
- Automatic routing between cloud and local models based on privacy setting
- Privacy mode indicator in UI

8.4 Safety Layer [CORE]

What it does: Validates all agent actions against a policy ruleset before execution, blocking harmful or out-of-scope operations.

Why it matters: Ensures GEWN cannot be manipulated into destructive behavior.

Implementation:

- Middleware validation pipeline (pre-execution hook)
- Configurable safety policy definitions
- Graceful failure with user-facing explanation

---

9. Developer Platform

9.1 GEWN API [FUTURE]

What it does: Exposes GEWN's core capabilities (chat, agents, memory, automation) via a documented REST and WebSocket API.

Why it matters: Enables third-party applications to embed GEWN intelligence.

Implementation:

- FastAPI backend with OpenAPI documentation
- API key management and rate limiting
- WebSocket endpoints for streaming responses

9.2 Agent SDK `[FUTURE]`

What it does: A developer toolkit for building, testing, and deploying custom AI agents within the GEWN ecosystem.

Why it matters: Lowers the barrier to extending GEWN's autonomous capabilities.

Implementation:

- Modular agent base class with lifecycle hooks
- CLI scaffolding tool for new agent creation
- Local test harness with mock tool environments

9.3 Command Framework `[CORE]`

What it does: A registry of reusable, composable automation commands that can be combined into higher-order workflows.

Why it matters: Provides a standardized building block system for workflow construction.

Implementation:

- Command registry with typed input/output schemas
- Execution engine with error handling, retries, and logging
- Visual command composer (drag-and-drop, future)

9.4 Local Development Environment [CORE]

What it does: A modular, containerized development setup for contributors building on or extending GEWN.

Why it matters: Lowers onboarding friction and ensures consistent development environments.

Implementation:

- Docker Compose stack with hot reload
- Service isolation per module (AI, memory, automation, voice)
- Local environment config with `.env` management

---

10. Future & Experimental Features

10.1 Emotion-Aware AI [FUTURE]

What it does: Detects stress indicators and emotional patterns from voice tone and interaction behavior. Adapts response style accordingly.

Why it matters: Creates an adaptive experience that responds to the user's state, not just their words.

Implementation: Voice sentiment analysis, prosody modeling, adaptive prompt injection

---

10.2 Predictive Assistance [FUTURE]

What it does: Anticipates likely next actions based on behavioral history and proactively offers assistance before the user asks.

Why it matters: Moves toward ambient, zero-friction AI — where help arrives before the need is verbalized.

Implementation: Sequence prediction models trained on workflow history

---

10.3 AR Assistant Mode [FUTURE]

What it does: Projects GEWN's interface and overlays into AR/XR environments.

Why it matters: Expands GEWN beyond the desktop into spatial computing interfaces.

Implementation: AR SDK integration (ARKit, OpenXR), spatial overlay rendering

---

10.4 Autonomous Long-Running Tasks [FUTURE]

What it does: Executes extended, multi-hour workflows autonomously in the background with periodic status reporting.

Why it matters: Enables true background AI operations — research marathons, codebase refactors, data pipelines.

Implementation: Distributed agent execution, persistent task queues (Celery / Redis), checkpoint/resume

---

10.5 AI Operating Layer [FUTURE]

What it does: Deep OS-level integration that positions GEWN as an intelligent coordination layer across all applications, processes, and data on the device.

Why it matters: Represents the long-term vision: an AI-native desktop OS experience.

Implementation: OS-level hooks, adaptive workflow systems, inter-process AI orchestration

---

Feature Summary Matrix

Domain	Core Features	Future Features
Conversational AI	Chat, Context Awareness, Agents, Multi-Agent	—
Voice	Wake Word, STT, TTS	Real-Time Voice Mode
Automation	File Mgmt, App Control, Browser, Workflows	—
Vision	Screen Understanding, OCR, Error Detection	Visual Overlays

Domain	Core Features	Future Features
Memory	Short-Term, Long-Term, Workflow Memory	AI Second Brain
Productivity	Workspace Setup, Tasks, File Org, Research	Analytics, Notifications, Meeting Assist
Integrations	Service Integrations, Command Framework	Plugin SDK, Marketplace, GEWN API
Security	Permissions, Encryption, Privacy Mode, Safety	—
Developer	Local Dev Environment	Agent SDK, GEWN API
Experimental	—	Emotion AI, Predictive, AR Mode, Long Tasks, AI OS Layer

---

Product Vision

GEWN is not a chatbot. It is an **AI-native operating layer** — a system that sees, remembers, decides, and acts alongside the user across every surface of their digital environment. The convergence of conversational AI, autonomous agents, persistent memory, voice interaction, and deep desktop integration represents a new category of software: the **personal AI copilot**.

The architecture is designed for progressive capability — each layer can operate independently, but the full system creates emergent intelligence greater than any individual component.

Document maintained by the GEWN core team. Feature scope and implementation details subject to change based on technical feasibility and user research.